

# MO89905: Homework 1

Md. Symum Hossain (ID: 20264789)  
Cho Chun Shik Graduate School of Mobility,  
Korea Advanced Institute of Science and Technology (KAIST)  
Fall 2026

## Problem 1: Part 1

### Bellman Recursion & Terminal Condition

#### A. Terminal Condition ( $k = 4$ )

$$J_4^*(S_4) = g_4(S_4)$$

For the completed request set  $S_4 = \{A, B, C, D\}$ , the terminal cost is zero:

$$J_4^*(\{A, B, C, D\}) = 0.$$

Here,

- $J_4^*(S_4)$ : Optimal Cost-to-Go
- $S_4$ : Terminal State
- $g_4(S_4)$ : Terminal Cost

#### B. Backward Bellman Recursion ( $k = 3, 2, 1$ )

$$J_k^*(S_k) = \min_{u_k \in \{A, B, C, D\} \setminus S_k} [c_k(u_k) + J_{k+1}^*(S_k \cup \{u_k\})].$$

Here,

- $J_k^*(S_k)$ : Optimal Cost-to-Go
- $S_k$ : Current State
- $u_k$ : Control operation
- $\{A, B, C, D\} \setminus S_k$ : Admissible Control Set
- $c_k(u_k)$ : Immediate Cost
- $S_k \cup \{u_k\}$ : Next State
- $J_{k+1}^*(S_k \cup \{u_k\})$ : Future Optimal Cost-to-Go

So the Bellman equation means:

$$\text{Optimal Cost-to-Go} = \text{Immediate Cost} + \text{Future Optimal Cost-to-Go}$$

where the action is chosen to minimize the total cost.

**Initial Stage ( $k = 0$ )**

For  $X \in \{P, Q, R\}$  and  $S_0 = \emptyset$ ,

$$J_0^{*,X}(\emptyset) = \min_{u_0 \in \{A,B,C,D\}} [c_0^X(u_0) + J_1^*(\{u_0\})].$$

Here,

- $X$ : Initial Operating Condition
- $S_0 = \emptyset$ : Initial State

**Corresponding optimal first action:**

$$u_0^{*,X} \in \arg \min_{u_0 \in \{A,B,C,D\}} [c_0^X(u_0) + J_1^*(\{u_0\})].$$

**Problem 1: Part 2**

I worked backwards from the terminal stage  $k = 4$ . At every state, the candidate value for selecting request  $u_k$  is:

$$c_k(u_k) + J_{k+1}^*(S_k \cup \{u_k\})$$

**Terminal Condition ( $k = 4$ ):** All four requests have been completed:

$$S_4 = \{A, B, C, D\}$$

and there is no terminal cost:

$$J_4^*(\{A, B, C, D\}) = 0$$

**Stage  $k = 3$ :**

At  $k = 3$ , three requests have already been completed, so only one request remains in each state.

**case 1: State  $S_3 = \{A, B, C\}$**  Only  $D$  can be selected:

$$D : \quad c_3(D) + J_4^*(\{A, B, C, D\}) = 6 + 0 = 6$$

Therefore:

$$J_3^*(\{A, B, C\}) = 6, \quad u_3^* = D$$

**case 2: State  $S_3 = \{A, B, D\}$**  Only  $C$  can be selected:

$$C : \quad c_3(C) + J_4^*(\{A, B, C, D\}) = 4 + 0 = 4$$

Therefore:

$$J_3^*(\{A, B, D\}) = 4, \quad u_3^* = C$$

**case 3: State  $S_3 = \{A, C, D\}$**  Only  $B$  can be selected:

$$B : \quad c_3(B) + J_4^*(\{A, B, C, D\}) = 2 + 0 = 2$$

Therefore:

$$J_3^*(\{A, C, D\}) = 2, \quad u_3^* = B$$

**case 4: State**  $S_3 = \{B, C, D\}$  Only  $A$  can be selected:

$$A : c_3(A) + J_4^*(\{A, B, C, D\}) = 5 + 0 = 5$$

Therefore:

$$J_3^*(\{B, C, D\}) = 5, \quad u_3^* = A$$

**Stage  $k = 2$ :**

Now two requests have already been completed. We use the  $J_3^*$  values calculated in the previous stage.

**case 1: State**  $S_2 = \{A, B\}$  Remaining requests:  $C, D$

$$C : c_2(C) + J_3^*(\{A, B, C\}) = 5 + 6 = 11$$

$$D : c_2(D) + J_3^*(\{A, B, D\}) = 2 + 4 = 6$$

Therefore:

$$J_2^*(\{A, B\}) = 6, \quad u_2^* = D$$

**case 2: State**  $S_2 = \{A, C\}$  Remaining requests:  $B, D$

$$B : c_2(B) + J_3^*(\{A, B, C\}) = 6 + 6 = 12$$

$$D : c_2(D) + J_3^*(\{A, C, D\}) = 2 + 2 = 4$$

Therefore:

$$J_2^*(\{A, C\}) = 4, \quad u_2^* = D$$

**case 3: State**  $S_2 = \{A, D\}$  Remaining requests:  $B, C$

$$B : c_2(B) + J_3^*(\{A, B, D\}) = 6 + 4 = 10$$

$$C : c_2(C) + J_3^*(\{A, C, D\}) = 5 + 2 = 7$$

Therefore:

$$J_2^*(\{A, D\}) = 7, \quad u_2^* = C$$

**case 4: State**  $S_2 = \{B, C\}$  Remaining requests:  $A, D$

$$A : c_2(A) + J_3^*(\{A, B, C\}) = 3 + 6 = 9$$

$$D : c_2(D) + J_3^*(\{B, C, D\}) = 2 + 5 = 7$$

Therefore:

$$J_2^*(\{B, C\}) = 7, \quad u_2^* = D$$

**case 5: State**  $S_2 = \{B, D\}$  Remaining requests:  $A, C$

$$A : c_2(A) + J_3^*(\{A, B, D\}) = 3 + 4 = 7$$

$$C : c_2(C) + J_3^*(\{B, C, D\}) = 5 + 5 = 10$$

Therefore:

$$J_2^*(\{B, D\}) = 7, \quad u_2^* = A$$

**case 6: State**  $S_2 = \{C, D\}$  Remaining requests:  $A, B$

$$A : c_2(A) + J_3^*(\{A, C, D\}) = 3 + 2 = 5$$

$$B : c_2(B) + J_3^*(\{B, C, D\}) = 6 + 5 = 11$$

Therefore:

$$J_2^*(\{C, D\}) = 5, \quad u_2^* = A$$

### Stage $k = 1$

Now only one request has been completed. We use the  $J_2^*$  values from the previous stage.

**case 1: State  $S_1 = \{A\}$**  Remaining requests:  $B, C, D$

$$B : 4 + J_2^*(\{A, B\}) = 4 + 6 = 10$$

$$C : 2 + J_2^*(\{A, C\}) = 2 + 4 = 6$$

$$D : 5 + J_2^*(\{A, D\}) = 5 + 7 = 12$$

Therefore:

$$J_1^*(\{A\}) = 6, \quad u_1^* = C$$

**case 2: State  $S_1 = \{B\}$**  Remaining requests:  $A, C, D$

$$A : 6 + J_2^*(\{A, B\}) = 6 + 6 = 12$$

$$C : 2 + J_2^*(\{B, C\}) = 2 + 7 = 9$$

$$D : 5 + J_2^*(\{B, D\}) = 5 + 7 = 12$$

Therefore:

$$J_1^*(\{B\}) = 9, \quad u_1^* = C$$

**case 3: State  $S_1 = \{C\}$**  Remaining requests:  $A, B, D$

$$A : 6 + J_2^*(\{A, C\}) = 6 + 4 = 10$$

$$B : 4 + J_2^*(\{B, C\}) = 4 + 7 = 11$$

$$D : 5 + J_2^*(\{C, D\}) = 5 + 5 = 10$$

There is a tie between  $A$  and  $D$ . By the alphabetical tie-breaking rule, select  $A$ .

$$J_1^*(\{C\}) = 10, \quad u_1^* = A$$

**case 4: State  $S_1 = \{D\}$**  Remaining requests:  $A, B, C$

$$A : 6 + J_2^*(\{A, D\}) = 6 + 7 = 13$$

$$B : 4 + J_2^*(\{B, D\}) = 4 + 7 = 11$$

$$C : 2 + J_2^*(\{C, D\}) = 2 + 5 = 7$$

Therefore:

$$J_1^*(\{D\}) = 7, \quad u_1^* = C$$

### Stage $k = 0$ : Three Initial Operating Conditions

At  $k = 0$ , the initial state is empty:

$$S_0 = \emptyset$$

and the first-request cost depends on the initial operating condition  $X \in \{P, Q, R\}$ . We use the Bellman equation:

$$J_0^{*,X}(\emptyset) = \min_{u_0} [c_0^X(u_0) + J_1^*(\{u_0\})].$$

**case 1: Initial Condition  $P$**

$$\begin{aligned} A : & 5 + J_1^*({A}) = 5 + 6 = 11 \\ B : & 1 + J_1^*({B}) = 1 + 9 = 10 \\ C : & 7 + J_1^*({C}) = 7 + 10 = 17 \\ D : & 6 + J_1^*({D}) = 6 + 7 = 13 \end{aligned}$$

Therefore:

$$J_0^{*,P}(\emptyset) = 10, \quad u_0^{*,P} = B$$

**case 2: Initial Condition  $Q$**

$$\begin{aligned} A : & 1 + J_1^*({A}) = 1 + 6 = 7 \\ B : & 6 + J_1^*({B}) = 6 + 9 = 15 \\ C : & 5 + J_1^*({C}) = 5 + 10 = 15 \\ D : & 4 + J_1^*({D}) = 4 + 7 = 11 \end{aligned}$$

Therefore:

$$J_0^{*,Q}(\emptyset) = 7, \quad u_0^{*,Q} = A$$

**case 3: Initial Condition  $R$**

$$\begin{aligned} A : & 6 + J_1^*({A}) = 6 + 6 = 12 \\ B : & 5 + J_1^*({B}) = 5 + 9 = 14 \\ C : & 2 + J_1^*({C}) = 2 + 10 = 12 \\ D : & 4 + J_1^*({D}) = 4 + 7 = 11 \end{aligned}$$

Therefore:

$$J_0^{*,R}(\emptyset) = 11, \quad u_0^{*,R} = D$$

The **Transition Diagram** for the Optimality is Given:

## Problem 1: Part 3:

### Optimal Request Order and Total Cost

Using the optimal decisions obtained from the backward DP table in Part 2, we follow the selected request at each successive state.

#### 1. Initial operating condition $P$

From Part 2:

$$u_0^{*,P} = B$$

the optimal request order is

$$B \rightarrow C \rightarrow D \rightarrow A$$

The total cost is

$$1 + 2 + 2 + 5 = 10.$$

Hence,

$$J_0^{*,P}(\emptyset) = 10$$

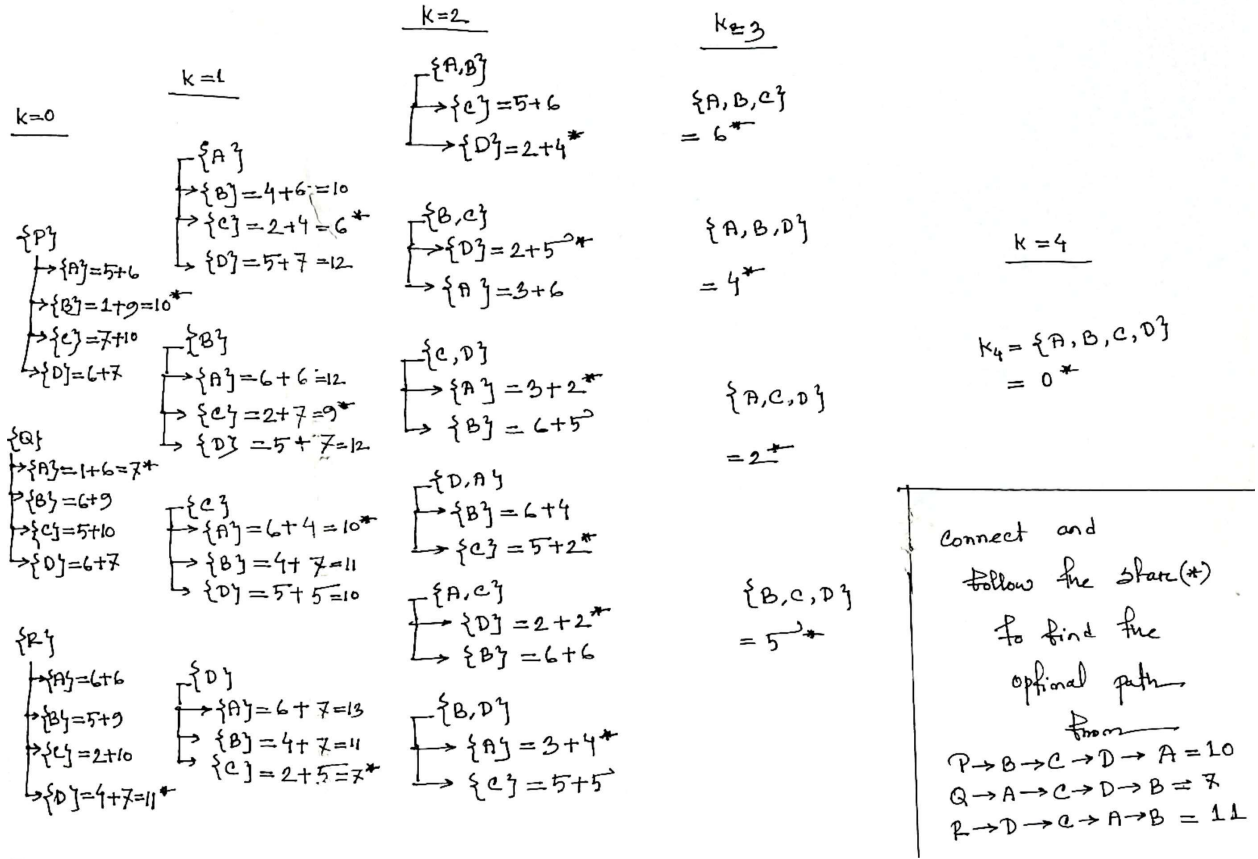


Figure 1: The Transition Flow Diagram

## 2. Initial operating condition $Q$

From Part 2:

$$u_0^{*,Q} = A$$

the optimal request order is

$$A \rightarrow C \rightarrow D \rightarrow B$$

The total cost is

$$1 + 2 + 2 + 2 = 7.$$

Hence,

$$J_0^{*,Q}(\emptyset) = 7$$

## 3. Initial operating condition $R$

From Part 2:

$$u_0^{*,R} = D$$

the optimal request order is

$$D \rightarrow C \rightarrow A \rightarrow B$$

The total cost is

$$4 + 2 + 3 + 2 = 11.$$

Hence,

$$J_0^{*,R}(\emptyset) = 11$$

## How One DP Table Supplies Optimal Continuations

The DP table stores the optimal cost-to-go  $J_k^*(S_k)$  for every possible state, not just for the initial state.

For example, once  $B$  is selected under initial condition  $P$ , the state becomes

$$S_1 = \{B\}.$$

The same precomputed value (from backward)

$$J_1^*(B) = 9$$

and its corresponding optimal action  $C$  are used to determine the continuation. From there, the table gives

$$\{B\} \rightarrow C$$

$$\{B, C\} \rightarrow D$$

and then

$$\{B, C, D\} \rightarrow A.$$

The important point is that the continuation depends only on the current state  $S_k$ , not on how that state was reached. Therefore, the same DP table can be reused for different initial conditions  $P, Q, R$ .

**One DP table  $\rightarrow$  optimal continuation from every state**

**This is the key advantage of Dynamic Programming and the Principle of Optimality.**

## Problem 2: Part 1:

### The Bellman Recursion

**Terminal condition** (the finish line), for every  $(e, \eta) \in \mathcal{X}_{12}$ :

$$J_{12}(e, \eta) = g_N(e, \eta) = 4e^2 + \eta^2$$

**Recursion**, for  $k = 11, 10, \dots, 0$  and every  $(e, \eta) \in \mathcal{X}_k$ :

$$J_k(e, \eta) = \min_{u \in \mathcal{U}(\eta)} \left[ e^2 + 0.25\eta^2 + 0.5u^2 + \sum_{w \in \{-1, 0, 1\}} p(w) J_{k+1} \left( \underbrace{e + \eta - \Delta r_k}_{e_{k+1}}, \underbrace{\eta + u + w}_{\eta_{k+1}} \right) \right] \quad (1)$$

Here  $p(-1) = 0.1$ ,  $p(0) = 0.8$ , and  $p(1) = 0.1$ .

The optimal control  $\mu_k(e, \eta)$  is the minimizing  $u$ . If several controls tie within  $10^{-12}$ , I pick the first one in the order  $-1, 0, 1$ .

The bracket is "**the stage cost + the expected optimal cost-to-go**". Also here,  $e_{k+1}$  depends on  $\eta_k$ , not on  $u_k$ . So, steering changes your drift, and the drift moves you one step later. A vehicle's lean takes a moment to change their path. So  $u$  affects the error only with a one-step delay.

The values of  $\Delta r_k = r_{k+1} - r_k$  for  $k = 0 \dots 11$  are:

$$\Delta r = (1, 1, 1, -1, -1, -1, -1, -1, -1, 1, 1, 1)$$

### Numerical Backup at $k = 11$ for State $(e, \eta) = (0, 0)$

We define two sets of lookup tables (matrices) for each time step  $k$ :

- **Optimal Value Table ( $J_k$ ):** Stores the minimum expected remaining cost from step  $k$  to the finish line for every valid state  $(e, \eta) \in \mathcal{X}_k$ .
- **Optimal Control Table ( $\mu_k$ ):** Stores the exact steering action  $u \in \{-1, 0, 1\}$  that minimizes that cost.

**Terminal Condition** ( $k = 12$ ): For every state  $(e, \eta) \in \mathcal{X}_{12}$ :

$$J_{12}(e, \eta) = 4e^2 + \eta^2$$

**Backward DP Recursion** (for  $k = 11, 10, \dots, 0$ ): seen in (1) where  $p(-1) = 0.1$ ,  $p(0) = 0.8$ , and  $p(1) = 0.1$ .

The optimal policy table is updated as:

$$\mu_k(e, \eta) = \arg \min_{u \in \mathcal{U}(\eta)} Q_k(e, \eta, u)$$

In case of ties within  $10^{-12}$ , the tie-breaking protocol chooses the first active control from the ordered array  $[-1, 0, 1]$ .

We evaluate the step-by-step arithmetic to fill the lookup table at step  $k = 11$  for the specific coordinate intersection  $(0, 0)$ .

**Step 1: Deterministic Error State Transition** The track change parameter at this step is  $\Delta r_{11} = 1$ . Due to the one-step physical delay, the next error state  $e_{12}$  depends purely on the current state variables and is independent of the control  $u$ :

$$e_{12} = e + \eta - \Delta r_{11} = 0 + 0 - 1 = -1$$

**Step 2: Admissible Controls** Since  $\eta = 0$ , the physical boundary condition  $|\eta + u| \leq 1$  evaluates to  $|0 + u| \leq 1$ . All three actions are mathematically legal:  $\mathcal{U}(0) = \{-1, 0, 1\}$ .

**Step 3: Action-Value  $Q(u)$  Calculation** The base terminal cost for landing at error  $e_{12} = -1$  simplifies the future cost lookup to:

$$J_{12}(-1, \eta_{12}) = 4(-1)^2 + \eta_{12}^2 = 4 + \eta_{12}^2$$

The total cost score  $Q(u) = g(e, \eta, u) + \mathbb{E}[J_{12}]$  is calculated for each control option below:

1. **For Control  $u = -1$ :**

- Stage Cost:  $g = 0^2 + 0.25(0)^2 + 0.5(-1)^2 = 0.5$
- Future Drift States ( $\eta_{12} = \eta + u + w$ ):
  - Wind  $w = -1 \implies \eta_{12} = 0 - 1 - 1 = -2 \implies J_{12} = 4 + (-2)^2 = 8$
  - Wind  $w = 0 \implies \eta_{12} = 0 - 1 + 0 = -1 \implies J_{12} = 4 + (-1)^2 = 5$
  - Wind  $w = 1 \implies \eta_{12} = 0 - 1 + 1 = 0 \implies J_{12} = 4 + (0)^2 = 4$
- Expected Future Cost:  $\mathbb{E}[J_{12}] = (0.1 \times 8) + (0.8 \times 5) + (0.1 \times 4) = 5.2$
- Total Score:  $Q(-1) = 0.5 + 5.2 = \mathbf{5.7}$

2. **For Control  $u = 0$ :**

- Stage Cost:  $g = 0^2 + 0.25(0)^2 + 0.5(0)^2 = 0$
- Future Drift States ( $\eta_{12} = \eta + u + w$ ):
  - Wind  $w = -1 \implies \eta_{12} = 0 + 0 - 1 = -1 \implies J_{12} = 4 + (-1)^2 = 5$
  - Wind  $w = 0 \implies \eta_{12} = 0 + 0 + 0 = 0 \implies J_{12} = 4 + (0)^2 = 4$
  - Wind  $w = 1 \implies \eta_{12} = 0 + 0 + 1 = 1 \implies J_{12} = 4 + (1)^2 = 5$
- Expected Future Cost:  $\mathbb{E}[J_{12}] = (0.1 \times 5) + (0.8 \times 4) + (0.1 \times 5) = 4.2$
- Total Score:  $Q(0) = 0 + 4.2 = \mathbf{4.2}$

3. **For Control  $u = 1$ :**

- Stage Cost:  $g = 0^2 + 0.25(0)^2 + 0.5(1)^2 = 0.5$



- Future Drift States ( $\eta_{12} = \eta + u + w$ ):
  - Wind  $w = -1 \implies \eta_{12} = 0 + 1 - 1 = 0 \implies J_{12} = 4 + (0)^2 = 4$
  - Wind  $w = 0 \implies \eta_{12} = 0 + 1 + 0 = 1 \implies J_{12} = 4 + (1)^2 = 5$
  - Wind  $w = 1 \implies \eta_{12} = 0 + 1 + 1 = 2 \implies J_{12} = 4 + (2)^2 = 8$
- Expected Future Cost:  $\mathbb{E}[J_{12}] = (0.1 \times 4) + (0.8 \times 5) + (0.1 \times 8) = 5.2$
- Total Score:  $Q(1) = 0.5 + 5.2 = \mathbf{5.7}$

**Step 4: Storing the optimal** Comparing the evaluated outcomes  $\{5.7, 4.2, 5.7\}$ , the minimum value is 4.2, corresponding to action  $u = 0$ . These exact outputs are written directly into the global matrices at index coordinate  $(0, 0)$ :

$$J_{11}(0, 0) = \mathbf{4.2}, \quad \mu_{11}(0, 0) = \mathbf{0}$$

**Implementation result.** Backward DP was implemented in Python for all 3185 states  $(k, e, \eta)$ ,  $k = 0, \dots, 12$ , storing the optimal value  $J_k(e, \eta)$  and the optimal control  $\mu_k(e, \eta)$  at every grid point (code: <https://github.com/SymumAust/Slalom/tree/main>). The resulting optimal cost from the initial state is

$$J_0^*(0, 0) = 29.135856, \quad \mu_0(0, 0) = +1.$$

```
PS D:\Slalom> & C:\Users\symum\AppData\Local\Python\pythoncore-3.14-64\python.exe
k=11: Delta r_11 = 1, e_12 = -1
u=-1: g=0.50, E[J12]=5.20, Q=5.70 [(-1, 0.1, 8), (0, 0.8, 5), (1, 0.1, 4)]
u=+0: g=0.00, E[J12]=4.20, Q=4.20 [(-1, 0.1, 5), (0, 0.8, 4), (1, 0.1, 5)]
u=+1: g=0.50, E[J12]=5.20, Q=5.70 [(-1, 0.1, 4), (0, 0.8, 5), (1, 0.1, 8)]
J_11(0,0) = 4.2000, mu_11(0,0) = 0
J_0(0,0) = 29.135856, mu_0(0,0) = 1
```

Figure 2: Output of backward DP.

## Reason of Specified Grids and Controls Need No Clipping.

The state space grids and controls are perfectly aligned with the system’s physics, meaning the vehicle **never exits the boundaries** (no clipping) and **always lands exactly on a grid point** (no interpolation) due to three facts:

- **Drift Bounds are Self-Correcting** ( $|\eta| \leq 2$ ): The control constraint strictly enforces that  $|\eta_k + u_k| \leq 1$ . Even when the maximum random wind disturbance ( $|w| = 1$ ) hits the vehicle, the next drift state can never exceed:

$$|\eta_{k+1}| \leq 1 + 1 \leq 2$$

If the drift reaches the boundary ( $\pm 2$ ), a valid counter-steering control ( $u = \mp 1$ ) always exists to pull it back toward safety.

- **The Error Fence Expands Fast Enough** ( $|e| \leq 6 + 3k$ ): The maximum distance the vehicle can physically slide sideways in a single step is  $|e_{k+1} - e_k| = |\eta_k - \Delta r_k|$ . Because the worst-case drift is  $|\eta_k| = 2$  and the sharpest track change is  $|\Delta r_k| = 1$ , the vehicle can move at most  $2 + 1 = 3$  units per step. Since the error grid  $\mathcal{X}_k$  is designed to expand its boundaries by exactly 3 units at each step (from  $\pm 6$  at  $k = 0$  out to  $\pm(6 + 3k)$ ), the vehicle can never physically catch up to or cross the boundary fence.
- **Pure Integer Dynamics (No Decimals)**: Every single variable that alters the states—the starting coordinates, the controls  $u \in \{-1, 0, 1\}$ , the track changes  $\Delta r_k \in \{-1, 1\}$ , and the random wind steps  $w \in \{-1, 0, 1\}$ —are strict integers. Adding integers always results in an integer, so the vehicle always lands precisely on a discrete grid intersection. No decimals are ever generated, completely eliminating the need for numerical interpolation.

These are the key specified reasons for the absence of clipping requirements.

## Problem 2: Part 2:

### Value Iteration

**Augmented states**  $z = (k, e, \eta)$ , where  $k = 0, \dots, 12$ ,  $e \in \{-6 - 3k, \dots, 6 + 3k\}$ , and  $\eta \in \{-2, \dots, 2\}$ . This gives:

$$\sum_{k=0}^{12} (13 + 6k) \cdot 5 = 3185 \text{ states} + \Delta \text{ absorbing state} = 3186 \text{ states}$$

### Bellman Optimality Operator:

$$(TJ)(z) = \begin{cases} \min_{u \in \mathcal{U}(\eta)} \left[ e^2 + 0.25\eta^2 + 0.5u^2 + \sum_w p(w)J(k+1, e + \eta - \Delta r_k, \eta + u + w) \right], & k < 12 \\ 4e^2 + \eta^2 + J(\Delta), & k = 12 \\ 0, & z = \Delta \end{cases}$$

Where,  $p(-1) = 0.1$ ,  $p(0) = 0.8$ ,  $p(1) = 0.1$ , and  $\mathcal{U}(\eta) = \{u \in \{-1, 0, 1\} : |\eta + u| \leq 1\}$ .

### Pseudo-code for the Algorithm:

---

#### Algorithm 1: Value Iteration for Bellman Optimality

---

```

1 Initialize  $J(k, e, \eta) = 0$  for every augmented state
2 Set sweep = 0
3 repeat
4   Copy the current table  $J$  into  $OLD$ 
5   foreach augmented state  $(k, e, \eta)$  do
6     if  $k = 12$  then
7        $NEW(k, e, \eta) = 4e^2 + \eta^2 + OLD(\Delta)$ 
8     else
9       foreach admissible control  $u \in \mathcal{U}(\eta)$  do
10        Calculate immediate cost:  $g(e, \eta, u) = e^2 + 0.25\eta^2 + 0.5u^2$ 
11        Calculate expected future cost using  $OLD$  values:
12         $\mathbb{E}[J] = \sum_w p(w) \cdot OLD(k+1, e + \eta - \Delta r_k, \eta + u + w)$ 
13        Calculate candidate = immediate cost +  $\mathbb{E}[J]$ 
14       $NEW(k, e, \eta) = \min_u(\text{candidate})$ 
15   residual =  $\max_{(k, e, \eta)} |NEW(k, e, \eta) - OLD(k, e, \eta)|$ 
16   Replace  $J$  with  $NEW$ 
17   sweep = sweep + 1
18 until residual  $< 10^{-10}$ 
19 Extract the greedy policy from the converged  $J$ 
20 Report the number of sweep

```

---

**Output:** Find the full code here: <https://github.com/SymumAust/Slalom/tree/main>

Number of sweeps = 14

**Converged values:**  $J(0, 0, 0) = 29.135856$

**Greedy policy along  $(e, \eta) = (0, 0)$ :**  $\mu(k, 0, 0) = (1, 0, 0, -1, -1, -1, -1, 0, 0, 1, 1, 0)$  for  $k = 0, \dots, 11$ .

## Stopping Criterion

Starting from  $J = 0$ , sweep 1 makes layer  $k = 12$  exact ( $J = g_N$ ). Each later sweep makes one more layer exact, working backward, because layer  $k$  is computed from layer  $k + 1$  of the old table, which is already exact. After **13 sweeps**, all 13 layers ( $k = 12$  down to 0) are exact, so  $J = J^*$ . **\*Sweep 14\*** applies  $T$  to  $J^*$ , **which changes nothing** because  $TJ^* = J^*$ . The residual is exactly  $0 < 10^{-10}$ , and the algorithm stops.

So the count is 13 sweeps to reach the solution plus 1 sweep to confirm it.

```

PS D:\Slalom> & C:\Users\symum\AppData\Local\Python\pythoncore-3.14-64\python.exe d
sweep 1: residual = 7.060000e+03
sweep 2: residual = 7.057200e+03
sweep 3: residual = 6.725500e+03
sweep 4: residual = 6.088620e+03
sweep 5: residual = 4.682156e+03
sweep 6: residual = 3.422433e+03
sweep 7: residual = 2.355769e+03
sweep 8: residual = 1.811635e+03
sweep 9: residual = 1.339410e+03
sweep 10: residual = 9.391743e+02
sweep 11: residual = 4.315794e+02
sweep 12: residual = 1.239836e+02
sweep 13: residual = 1.249815e+01
sweep 14: residual = 0.000000e+00

Number of augmented states: 3185 (+ DELTA)
Number of sweeps: 14

```

Figure 3: Output of VI

## Problem 2: Part 3:

### Policy Iteration

Augmented states are  $z = (k, e, \eta)$  with:

$$k = 0, \dots, 12, \quad e \in \{-6 - 3k, \dots, 6 + 3k\}, \quad \text{and} \quad \eta \in \{-2, \dots, 2\}$$

plus the absorbing state  $\Delta$  with  $J(\Delta) = 0$ . Decisions are made at the 2760 states with  $k < 12$ .

Initial policy:

$$\mu_0(k, e, \eta) = -\text{sign}(\eta)$$

|              |    |    |   |    |    |
|--------------|----|----|---|----|----|
| $\eta$       | -2 | -1 | 0 | +1 | +2 |
| $u$          | +1 | +1 | 0 | -1 | -1 |
| $ \eta + u $ | 1  | 0  | 0 | 0  | 1  |

where  $\text{sign}(0) = 0$ .

Every  $|\eta + u| \leq 1$ , so the policy is feasible.

**Policy evaluation (exact backward pass, controls fixed):**

$$J_\mu(12, e, \eta) = 4e^2 + \eta^2 + J(\Delta)$$

$$J_\mu(k, e, \eta) = g(e, \eta, \mu(z)) + \sum_w p(w) J_\mu(k+1, e + \eta - \Delta r_k, \eta + \mu(z) + w), \quad k = 11, \dots, 0$$

There is no minimum over controls: each state uses only the control  $\mu(z)$ . One backward pass gives the exact cost of the policy.

### Policy improvement (all states)

$$\mu_{\text{new}}(z) = \arg \min_{u \in \mathcal{U}(\eta)} \left[ g(e, \eta, u) + \sum_w p(w) J_\mu(k+1, e + \eta - \Delta r_k, \eta + u + w) \right]$$

Ties within  $10^{-12}$  go to the first control in the order  $-1, 0, 1$ .

**Worked improvement step example(round 1, state (10,0,0))** The initial policy gives  $u = 0$  here. Since  $\Delta r_{10} = 1$ , the next error is  $e_{11} = 0 + 0 - 1 = -1$ . Evaluating the initial policy at stage 11 gives

$$J_\mu(11, -1, \eta) = 67.7, 37.95, 17.2, 5.95, 3.7 \quad \text{for } \eta = -2, -1, 0, 1, 2.$$

The admissible controls are  $u \in \{-1, 0, 1\}$ . Using the evaluated stage-11 values, the expected future cost for each candidate control is calculated using the disturbance probabilities  $P(-1) = 0.1$ ,  $P(0) = 0.8$ , and  $P(1) = 0.1$ .

For  $u = -1$ :

$$Q(-1) = 0.1(67.7) + 0.8(37.95) + 0.1(17.2) + 0.5 = 39.35.$$

For  $u = 0$ :

$$Q(0) = 0.1(37.95) + 0.8(17.2) + 0.1(5.95) = 18.15.$$

For  $u = +1$ :

$$Q(+1) = 0.1(17.2) + 0.8(5.95) + 0.1(3.7) + 0.5 = 7.35.$$

Comparing the candidate costs,  $Q(+1)$  is the minimum. Therefore, the policy is improved at this state from  $u = 0$  to  $u = +1$ :

$$\mu^{(0)}(10, 0, 0) = 0 \quad \longrightarrow \quad \mu^{(1)}(10, 0, 0) = +1.$$

Although  $u = +1$  incurs an immediate control penalty of 0.5, it produces a substantially lower expected future cost. This illustrates that policy improvement considers both immediate and future costs when selecting an action. Similarly the outcome is,

| Round | $J_\mu(0, 0, 0)$ | Controls changed     |
|-------|------------------|----------------------|
| 1     | 59.650000        | 896                  |
| 2     | 40.281717        | 43                   |
| 3     | 31.457664        | 12                   |
| 4     | 29.135856        | 2                    |
| 5     | 29.135856        | 0 $\rightarrow$ stop |

Number of evaluation/improvement rounds = 5

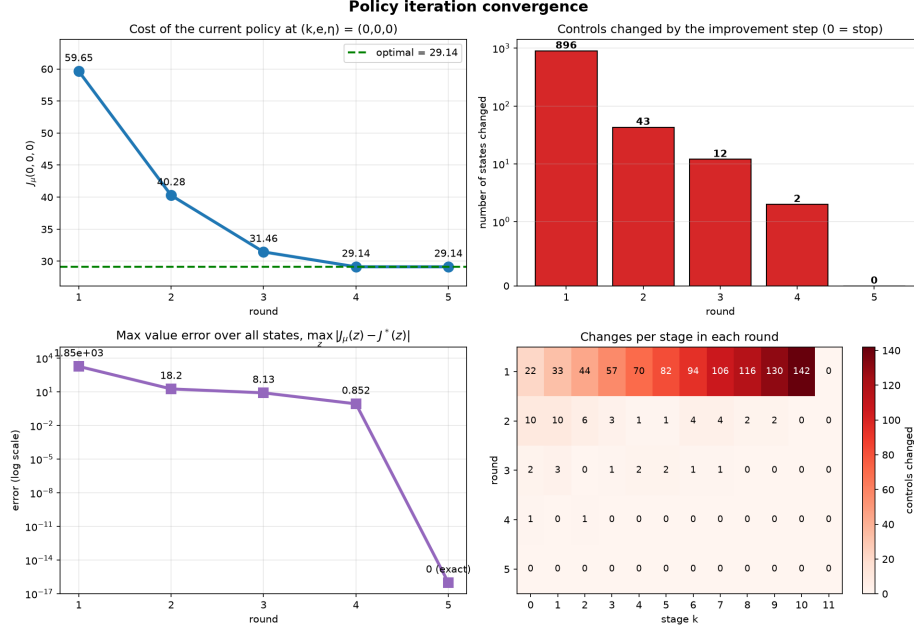


Figure 4: Policy Iteration Convergence

These values and the policy are identical to backward DP (Part 1) and value iteration (Part 2).  
Find the code here: <https://github.com/SymumAust/Slalom/tree/main>

## Problem 2: Part 4: Comparison & Explanation

### 4(a): Comparison of three methods

**Backward DP**, **value iteration**, and **policy iteration** were applied to the same problem. Backward DP solves the finite-horizon problem exactly, so it was used as the reference. For each method:

- **Initial values:** Read off as  $J_0^*(e, \eta) = J(0, e, \eta)$ .
- **Maximum value difference:** Computed as  $\max_z |J(z) - J_{DP}(z)|$  over all 3185 augmented states  $(k, e, \eta)$  and the absorbing state  $\Delta$ .
- **Policy disagreements:** The number of decision states ( $k \leq 11$ , 2760 states) where the control differs from the backward DP control.
- **Iteration count:** One backward pass for DP, synchronous sweeps for VI (stopping at  $\|TJ - J\|_\infty < 10^{-10}$ ), and evaluation/improvement rounds for PI.
- **Computation time:** Measured with `time.perf_counter()` around the solver only, excluding plotting, and reported as the median of 7 runs.

Table 1: Comparison of Backward DP, Value Iteration, and Policy Iteration

| Method           | $J_0^*(-2, 0)$ | $J_0^*(0, 0)$ | $J_0^*(2, 0)$ | Max. diff.<br>from DP | Policy<br>disagreements | Iterations      | Time<br>(ms) |
|------------------|----------------|---------------|---------------|-----------------------|-------------------------|-----------------|--------------|
| Backward DP      | 56.836609      | 29.135856     | 26.800816     | 0                     | 0                       | 1 backward pass | 9.24         |
| Value iteration  | 56.836609      | 29.135856     | 26.800816     | 0                     | 0                       | 14 sweeps       | 86.23        |
| Policy iteration | 56.836609      | 29.135856     | 26.800816     | 0                     | 0                       | 5 rounds        | 49.86        |

Note: Times are machine dependent.

## Understandings

All three methods give identical optimal values at every state, with a maximum difference of exactly 0. They also give identical policies, with 0 disagreements out of 2760 decision states. The agreement is exact because all successors are integer grid points, so no clipping or interpolation error is introduced. The policies match because all methods use the same tie-breaking rule (first control in the order  $-1, 0, 1$  within  $10^{-12}$  of the minimum).

The initial values are not symmetric in  $e$ . Since the reference first moves right ( $\Delta r_0 = +1$ ), a start at  $e = -2$  lets the error grow to  $-3$  before it can be corrected, giving 56.84. A start at  $e = +2$  lets the reference move toward the vehicle, giving 26.80.

```
PS D:\Slalom> & C:\Users\symum\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/Slalom/part4_compare.py
(a) Comparison table
Method      J0(-2,0)   J0(0,0)    J0(2,0)    max|J-J_DP|  disagree  iterations  time ms
Backward DP  56.836609  29.135856  26.800816  0.00e+00     0         1 backward pass  9.24
Value iteration  56.836609  29.135856  26.800816  0.00e+00     0         14 sweeps      86.23
Policy iteration  56.836609  29.135856  26.800816  0.00e+00     0         5 rounds       49.86

Work (Q evaluations): DP = 4968, VI = 14x4968 = 69552 (+4968 extraction), PI = 5x(2760+4968) = 38640

(b) VI error per sweep: ['8.83e+03', '7.53e+03', '6.09e+03', '4.68e+03', '3.42e+03', '2.36e+03', '1.81e+03', '1.34e+03', '939',
432', '124', '12.5', '0', '0']
    PI error per evaluation: ['1.85e+03', '18.2', '8.13', '0.852', '0']

(c) w = [0, 1, 0, 0, -1, 0, 0, 1, 0, 0, -1, 0]
Backward DP    u=[1, -1, -1, -1, 0, 1, 1, 0, -1, 0, 1, 0]  y=[0, 0, 1, 2, 2, 1, -1, -2, -2, -1, -1, -1]  cost=20.25
Value iteration u=[1, -1, -1, -1, 0, 1, 1, 0, -1, 0, 1, 0]  y=[0, 0, 1, 2, 2, 1, -1, -2, -2, -1, -1, -1]  cost=20.25
Policy iteration u=[1, -1, -1, -1, 0, 1, 1, 0, -1, 0, 1, 0]  y=[0, 0, 1, 2, 2, 1, -1, -2, -2, -1, -1, -1]  cost=20.25
stage costs: [0.5, 1.75, 1.75, 1.5, 0.25, 1.5, 1.75, 1.0, 0.75, 4.0, 1.5, 0.0]  terminal: 4  total: 20.25
J0*(0,0) = 29.135856
exact average over all 3^12 sequences = 29.135856
cheapest run = 8.0, most expensive run = 3751.0, P(cost <= 20.25) = 0.368
```

Figure 5: Comparison Output between three methods

The methods differ only in computational effort:

- **Backward DP** is fastest. It exploits the finite-horizon structure and computes each stage exactly once.
- **Value iteration** needs 14 sweeps: 13 to propagate exact values from  $k = 12$  back to  $k = 0$ , and one more to confirm a zero residual.
- **Policy iteration** needs 5 rounds: 896, 43, 12, and 2 controls change in the first four rounds, and none in the fifth. It uses fewer iterations than VI, but each round includes both an exact evaluation and a full improvement, so its time advantage over VI is smaller than the ratio of iteration counts suggests.

## 4(b): Value error versus iteration

**Error measure.** Backward DP (Part 1) gives the exact optimal cost-to-go  $J_{DP}$  on every augmented state. It is therefore used as the reference, and the error of any value table  $J$  is measured in the sup-norm over all states and stages,

$$\varepsilon(J) = \|J - J_{DP}\|_{\infty} = \max_{z \in \mathcal{Z}} |J(z) - J_{DP}(z)|, \quad (2)$$

where  $\mathcal{Z}$  contains the 3185 states  $z = (k, e, \eta)$ ,  $k = 0, \dots, 12$ , together with the absorbing state  $\Delta$ . For value iteration (VI) the error is recorded after every synchronous sweep  $J_s = T J_{s-1}$ . For policy iteration (PI) it is recorded after every exact policy evaluation, i.e.  $\varepsilon(J_{\mu_i})$  for the policy  $\mu_i$  of round  $i$ .

**Results.** Table 2 and Figure 6 show the error history. Exact zeros cannot be drawn on a logarithmic axis, so they are plotted at  $10^{-16}$ .

| Iteration | 1    | 2    | 3    | 4     | 5        | 6        | 7    |
|-----------|------|------|------|-------|----------|----------|------|
| VI        | 8830 | 7530 | 6090 | 4680  | 3420     | 2360     | 1810 |
| PI        | 1850 | 18.2 | 8.13 | 0.852 | <b>0</b> | –        | –    |
| Iteration | 8    | 9    | 10   | 11    | 12       | 13       | 14   |
| VI        | 1340 | 939  | 432  | 124   | 12.5     | <b>0</b> | 0    |

Table 2: Maximum value error  $\|J - J_{\text{DP}}\|_{\infty}$  after each VI sweep and after each PI policy evaluation.

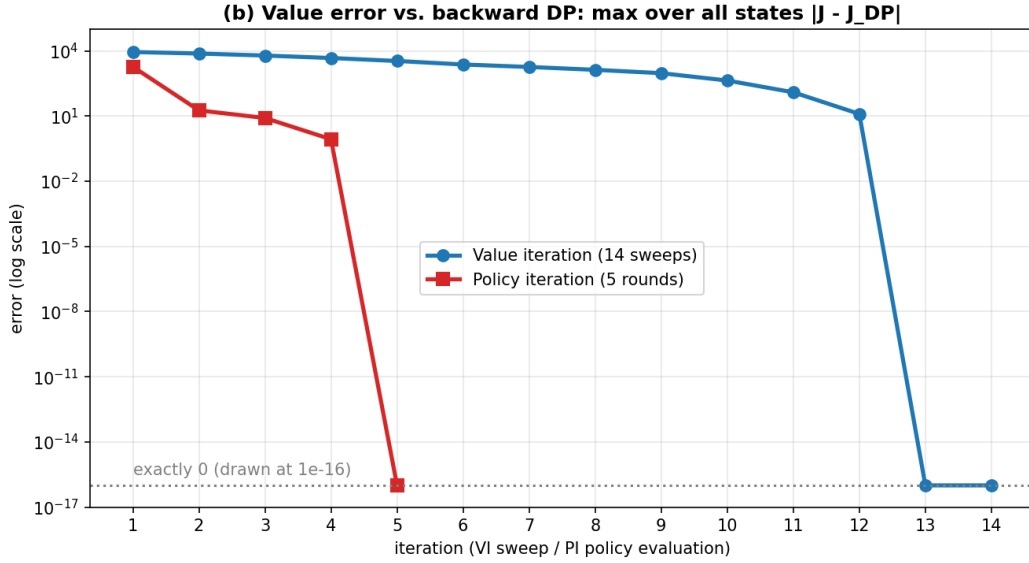


Figure 6: Value error relative to backward DP versus iteration (logarithmic scale). VI: error after each sweep. PI: error after each policy evaluation.

**Interpretation of the VI curve.** Starting from  $J_0 \equiv 0$ , sweep 1 makes layer  $k = 12$  exact ( $J = g_N$ ), and sweep  $s$  makes layers  $k = 13 - s, \dots, 12$  exact, since each layer is computed from the already exact layer behind it. The error is a maximum over *all* states, so it stays large until the last layer  $k = 0$  is exact. This happens at sweep 13, where the error becomes exactly 0; sweep 14 only confirms  $TJ = J$  (Bellman residual  $0 < 10^{-10}$ ). Because  $J_0 \equiv 0$  and all costs are nonnegative, VI approaches  $J^*$  from *below*: its early values account for only the last few stages of cost. For example, after sweep 1 the state  $(9, -33, -2)$  has value  $33^2 + 0.25 \cdot 4 + 0.5 = 1090.5$ , while  $J^*(9, -33, -2) = 9919.99$ , which gives the error  $\approx 8830$ .

**Interpretation of the PI curve.** Each PI value table is the exact cost of a feasible policy, so  $J_{\mu_i} \geq J^*$  and PI approaches  $J^*$  from *above*. The initial policy  $u = -\text{sign}(\eta)$  reacts only to the drift and ignores the reference, so its error is largest where a position correction is needed but no drift exists, e.g.  $J_{\mu_1}(3, 15, 0) = 4366.4$  versus  $J^*(3, 15, 0) = 2517.35$  (error  $\approx 1850$ ). By the policy improvement theorem,  $J_{\mu_{i+1}} \leq J_{\mu_i}$  at every state, and the error drops sharply ( $1850 \rightarrow 18.2 \rightarrow 8.13 \rightarrow 0.852 \rightarrow 0$ ). It reaches zero after the fifth evaluation, when no control changes.

**Why fewer iterations does not imply less computation.** PI needs 5 iterations and VI needs 14, but an iteration is not a fixed unit of work. Let one  $Q$ -evaluation denote the basic operation  $Q(z, u) = g(e, \eta, u) + \sum_w p(w) J(k+1, e + \eta - \Delta r_k, \eta + u + w)$ . Since  $|\mathcal{U}(\eta)| = 1, 2, 3, 2, 1$  for  $\eta = -2, \dots, 2$ , a full

Bellman backup over the  $552 \cdot 5 = 2760$  decision states ( $k \leq 11$ ) costs  $9 \cdot 552 = 4968$   $Q$ -evaluations. A policy evaluation uses only the fixed control and costs 2760.

| Method           | Work per iteration   | Iterations | Total $Q$ -evaluations | Time (ms) |
|------------------|----------------------|------------|------------------------|-----------|
| Backward DP      | 4968                 | 1          | 4,968                  | 9.24      |
| Value iteration  | 4968                 | 14         | 69,552                 | 86.23     |
| Policy iteration | $2760 + 4968 = 7728$ | 5          | 38,640                 | 49.86     |

Table 3: Computational work.

Hence the iteration count alone is not a measure of computational cost; the fair comparison is total work, i.e. (number of iterations)  $\times$  (cost per iteration), or measured run time.

#### 4(c): Simulating and realizing total cost differ

**Procedure.** Each feedback policy (DP, VI, PI) was simulated from  $x_0 = (e_0, \eta_0) = (0, 0)$  with the disturbance sequence  $(w_0, \dots, w_{11}) = (0, 1, 0, 0, -1, 0, 0, 1, 0, 0, -1, 0)$ . At each stage  $k$ :

- The control  $u_k = \mu_k(e_k, \eta_k)$  was chosen from the current state only.
- The stage cost  $g(e_k, \eta_k, u_k)$  was charged.
- The disturbance  $w_k$  was then applied:

$$e_{k+1} = e_k + \eta_k - \Delta r_k \quad (3)$$

$$\eta_{k+1} = \eta_k + u_k + w_k \quad (4)$$

The terminal cost  $g_N(e_{12}, \eta_{12})$  was added at the end. The disturbance sequence was used only to generate the state transitions; the controller never had access to future disturbances. **Results.** All three policies are identical (Part 4a), so they produce the same trajectory, shown in Table: 4 and Figure: 7.

Table 4: Simulation Trajectory Results

| $k$      | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    | 10   | 11   | 12   |
|----------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| $w_k$    | 0    | 1    | 0    | 0    | -1   | 0    | 0    | 1    | 0    | 0    | -1   | 0    | -    |
| $e_k$    | 0    | -1   | -1   | -1   | 0    | 0    | -1   | -1   | 0    | 2    | 1    | 0    | -1   |
| $\eta_k$ | 0    | 1    | 1    | 0    | -1   | -2   | -1   | 0    | 1    | 0    | 0    | 0    | 0    |
| $u_k$    | 1    | -1   | -1   | -1   | 0    | 1    | 1    | 0    | -1   | 0    | 1    | 0    | -    |
| $y_k$    | 0    | 0    | 1    | 2    | 2    | 1    | -1   | -2   | -2   | -1   | -1   | -1   | -1   |
| $r_k$    | 0    | 1    | 2    | 3    | 2    | 1    | 0    | -1   | -2   | -3   | -2   | -1   | 0    |
| Cost     | 0.50 | 1.75 | 1.75 | 1.50 | 0.25 | 1.50 | 1.75 | 1.00 | 0.75 | 4.00 | 1.50 | 0.00 | 4.00 |

#### Stage-cost computation

Each stage cost is  $g(e_k, \eta_k, u_k) = e_k^2 + 0.25 \eta_k^2 + 0.5 u_k^2$ , and the terminal cost is  $g_N(e_{12}, \eta_{12}) = 4e_{12}^2 + \eta_{12}^2$ .

Terminal cost at  $k = 12$ , with  $(e_{12}, \eta_{12}) = (-1, 0)$ :

$$g_N = 4(-1)^2 + 0^2 = 4.00$$

Realized total cost:

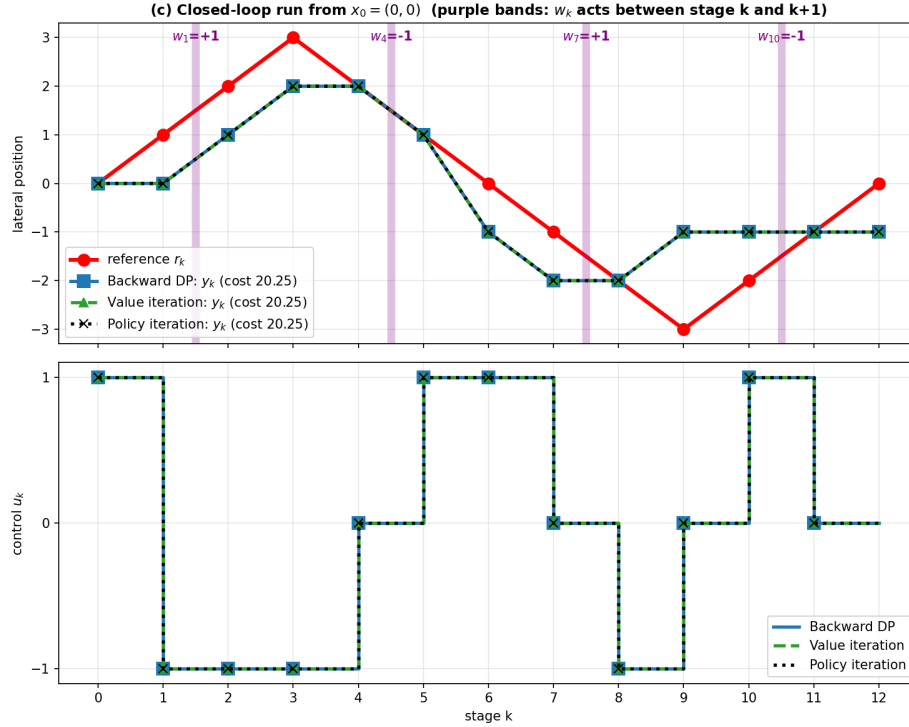
$$\sum_{k=0}^{11} g_k + g_N = 16.25 + 4.00 = 20.25$$

The realized total cost is 16.25 (stage costs) + 4.00 (terminal cost) = 20.25 for DP, VI and PI alike.



Table 5: Stage Cost Breakdown

| $k$ | $e_k$ | $\eta_k$ | $u_k$ | $e_k^2$ | $0.25 \eta_k^2$ | $0.5 u_k^2$ | $g_k$ |
|-----|-------|----------|-------|---------|-----------------|-------------|-------|
| 0   | 0     | 0        | 1     | 0       | 0               | 0.5         | 0.50  |
| 1   | -1    | 1        | -1    | 1       | 0.25            | 0.5         | 1.75  |
| 2   | -1    | 1        | -1    | 1       | 0.25            | 0.5         | 1.75  |
| 3   | -1    | 0        | -1    | 1       | 0               | 0.5         | 1.50  |
| 4   | 0     | -1       | 0     | 0       | 0.25            | 0           | 0.25  |
| 5   | 0     | -2       | 1     | 0       | 1               | 0.5         | 1.50  |
| 6   | -1    | -1       | 1     | 1       | 0.25            | 0.5         | 1.75  |
| 7   | -1    | 0        | 0     | 1       | 0               | 0           | 1.00  |
| 8   | 0     | 1        | -1    | 0       | 0.25            | 0.5         | 0.75  |
| 9   | 2     | 0        | 0     | 4       | 0               | 0           | 4.00  |
| 10  | 1     | 0        | 1     | 1       | 0               | 0.5         | 1.50  |
| 11  | 0     | 0        | 0     | 0       | 0               | 0           | 0.00  |
| Sum |       |          |       | 10      | 2.5             | 3.75        | 16.25 |

Figure 7: Closed-loop simulation from  $x_0 = (0, 0)$ 

The output  $y_k$  tracks  $r_k$  with roughly a one-step lag, because  $u_k$  acts on the drift and the drift affects the position one stage later. A disturbance  $w_k$  therefore first shows up in  $y_{k+2}$ . The policy cuts the sharp corners at  $k = 3$  and  $k = 9$  rather than paying extra drift and steering cost to reach them. It also anticipates the reversals of the reference: it steers left at  $k = 1-3$ , before the reference turns at  $k = 3$ , and steers right at  $k = 5-6$ , before the turn at  $k = 9$ .

**Why the realized cost differs from  $J_0^*(0, 0)$ .**  $J_0^*(0, 0) = 29.14$  is the expected cost, averaged over all  $3^{12} = 531,441$  disturbance sequences and weighted by their probabilities. The simulation uses one fixed sequence, so 20.25 is a single realization of a random cost, not its mean.

Simulating the optimal policy on every sequence confirms this. The probability-weighted average is

exactly 29.135856, and individual costs range from 8.0 to 3751.0. About 37% of sequences give a cost of at most 20.25, and the disturbance-free run costs 17.0. Because the cost is quadratic in the error, rare sequences with several same-direction disturbances are very expensive and pull the mean above the cost of a typical run. In the given sequence the disturbances alternate in sign and partly cancel, adding only about 3.25 to the disturbance-free cost.

The optimal policy minimizes the expected cost; it does not minimize the cost of every individual realization.

#### 4(d): Why the Three Methods Agree & differences

The three methods solve the exact same problem: they share identical grids, dynamics, costs, disturbance probabilities, and admissible controls.

The augmented state  $z = (k, e, \eta)$  with the zero-cost absorbing state  $\Delta$  used by Value Iteration (VI) and Policy Iteration (PI) is merely a rewriting of the finite-horizon problem as a stationary one; it is not a different model. All three methods compute the solution to the same Bellman equation:

$$J^* = TJ^* \quad (5)$$

Since every transition moves strictly from stage  $k$  to  $k+1$  until the absorbing state  $\Delta$  is reached, this equation is determined uniquely layer by layer, starting from the terminal condition  $J^*(N, \cdot) = g_N$ . Consequently, there is only one unique optimal value function.

- **Backward Dynamic Programming (DP)** solves the equation directly, processing one layer at a time.
- **Value Iteration (VI)** reaches the solution by applying the Bellman operator  $T$  repeatedly. One additional layer becomes exact with each sweep, meaning it becomes perfectly exact after 13 sweeps.
- **Policy Iteration (PI)** reaches the solution through a sequence of policies with non-increasing costs, stopping when  $TJ_\mu = J_\mu$ , which implies  $J_\mu = J^*$ .

The agreement across all methods is exact (**maximum difference of 0**) because every successor state lands precisely on an integer grid point, completely eliminating clipping or interpolation errors. The chosen policies also match perfectly (**0 disagreements**) because all methods share the same tie-breaking rule: they select the first control in the ordered sequence  $[-1, 0, 1]$  that falls within  $10^{-12}$  of the minimum value.

#### Bellman Backup vs. Policy Evaluation

Both operations are constructed from the exact same fundamental quantity, "cost now + expected cost-to-go":

$$Q(J, z, u) = g(e, \eta, u) + \sum(p * J[(k+1, e+\eta-dr[k], \eta+u+w)] \text{ for } w, p \text{ in } W)$$

##### Bellman Backup

Used in `bellman_T` within VI, as well as the **greedy** step in DP, VI policy extraction, and PI improvement:

$$TJ[z] = \min(Q(J, z, u) \text{ for } u \text{ in } \text{admissible}(\eta))$$

This operator tests all admissible controls and retains the cheapest option, yielding the best achievable cost starting from state  $z$ .

## Policy Evaluation

Used in `evaluate_policy` within PI:

$$J[z] = Q(J, z, \mu[z])$$

This operator restricts itself strictly to the control prescribed by the current policy, yielding the total expected cost of following that specific policy  $\mu$  from state  $z$ .

Table 6: Comparison Between Bellman Backup and Policy Evaluation

|                              | <b>Bellman Backup</b>             | <b>Policy Evaluation</b>            |
|------------------------------|-----------------------------------|-------------------------------------|
| <b>Question it answers</b>   | What's the best cost from here?   | What does this plan cost from here? |
| <b>Controls tried</b>        | All allowed (1–3 per state)       | Only the plan's one                 |
| <b>Uses min?</b>             | Yes                               | No                                  |
| <b>Example at</b> (10, 0, 0) | $\min(39.35, 18.15, 7.35) = 7.35$ | $Q(z, 0) = 18.15$                   |
| <b>Converges to</b>          | $J^*$ (optimal)                   | $J_\mu \geq J^*$ (one plan's cost)  |
| <b>Work per pass</b>         | 4,968                             | 2,760                               |
| <b>Used in</b>               | DP, VI, PI's improvement step     | PI's evaluation step                |

Policy Iteration systematically alternates between these two operations: it evaluates the current policy, then improves it by performing a greedy Bellman backup. In one line: the Bellman backup chooses the best control, while policy evaluation follows a given control. Everything else in the table follows from that one difference.

## References

- [1] K. Choi, *Learning-Based Control for Mobility Systems: Approximate Dynamic Programming, Reinforcement Learning, and Online Lookahead*, Chapter 2, Part II: “Exact and Approximate Dynamic Programming,” Sections 2.6–2.11, working draft v0.9.0-draft, Korea Advanced Institute of Science and Technology (KAIST), 12 Sep. 2026. Available: <https://kyunghwan-choi.github.io/learning-based-control-mobility/learning-based-control-mobility-ch02-part2-v0.9.0-draft.pdf>.
- [2] OpenAI, *ChatGPT* (GPT-5.6 Luna, default) [Large language model], 2026. Available: <https://chatgpt.com>.
- [3] Google, *Gemini* (default model, via Google AI) [Large language model], 2026. Available: <https://gemini.google.com>.
- [4] Anthropic, *Claude* (Claude Opus 5.5) [Large language model], 2026. Available: <https://claude.ai>.

## Statement of AI Tool Usage

| Tool                                | Provider  | Purpose in this report                                                                                                          |
|-------------------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------|
| ChatGPT (GPT-5.6 Luna, default) [2] | OpenAI    | Explanation of the problem statement and conceptual understanding of dynamic programming, value iteration and policy iteration. |
| Gemini (default, Google AI) [3]     | Google    | Report writing, LaTeX formatting and document arrangement.                                                                      |
| Claude (Opus 5.5) [4]               | Anthropic | Python implementation, visualization of results, and drafting of explanations text for Problem 2. (figures and diagrams).       |

Table 7: AI tools used in the preparation of this report.

*All AI-generated content was reviewed, and all numerical results were reproduced by running the code locally. The author takes full responsibility for the content of this report.*

*Simply open the code using VS code and click the run button; follow the output snapshots attached to the PDF for precise file location and all the related images and files are available at <https://github.com/SymumAust/Slalom/tree/main>.*